

final-melhorado-v2

June 12, 2026

```
[1]: '''  
      Bibliotecas usadas  
      '''  
  
      import os  
      from datetime import datetime  
      import re  
      import subprocess  
      import warnings  
      import numpy as np  
      import pandas as pd  
      from collections import Counter  
      from Bio import Entrez  
      from Bio import SeqIO  
      from Bio.Seq import Seq  
      from sklearn.model_selection import GroupShuffleSplit, StratifiedGroupKFold  
      from sklearn.metrics import (  
          classification_report,  
          roc_auc_score,  
          average_precision_score,  
          make_scorer,  
          matthews_corrcoef,  
          f1_score,  
          precision_score,  
          recall_score,  
          balanced_accuracy_score,  
          accuracy_score,  
          confusion_matrix,  
          ConfusionMatrixDisplay,  
          roc_curve,  
          auc  
      )  
  
      from sklearn.ensemble import RandomForestClassifier  
      from sklearn.model_selection import cross_validate  
      from sklearn.svm import SVC  
      from sklearn.cluster import KMeans
```

```

from sklearn.preprocessing import StandardScaler
from xgboost import XGBClassifier
import matplotlib.pyplot as plt
import subprocess
import shap
import seaborn as sns
from sklearn.inspection import permutation_importance

warnings.filterwarnings("ignore")

print("Bibliotecas importadas")

```

Bibliotecas importadas

```

[2]: '''
    obtenção dos datasets
    '''

EMAIL = "marcela.leite@gmail.com.br"
OUTPUT_DIR = "pipelineAB"
os.makedirs(OUTPUT_DIR, exist_ok=True)
Entrez.email = EMAIL

# =====
# QUERIES
# =====

# POSITIVE_QUERY = r'''
# ("Solanum lycopersicum"[Organism] OR "Nicotiana benthamiana"[Organism] OR
# ↪ "Capsicum annuum"[Organism] OR "Arabidopsis thaliana"[Organism])
# AND( TMV OR ToMV OR ToBRFV OR PMMoV OR TMGMV OR PVY OR PepYMV OR TEV OR PVA
# ↪ OR PVX OR TSWV OR GRSV OR TCSV OR CMV OR ToCV OR TICV OR AMV
# OR "virus resistance" OR "antiviral" OR "RNA silencing" OR "RDR" OR
# ↪ "Dicer-like" OR "Argonaute" OR "AGO" OR "eIF4E" OR "R gene" OR "Sw-5" OR
# ↪ "Tm-2" OR "RCY1"
# OR "Rx" OR "R protein" OR "NLR" OR "NBS-LRR" OR "NB-LRR" OR "TIR-NBS-LRR"
# ↪ OR "CC-NBS-LRR" OR "RLK" OR "RLP" OR "LRR receptor kinase"
# OR "leucine rich repeat receptor kinase" OR "plant disease resistance
# ↪ protein" OR "disease resistance protein" OR "RPM1" OR "RPS2" OR "RPS4" OR
# ↪ "RPS5"
# OR "RPP" OR "SNC1" OR "ADR1" OR "NDR1" OR "EDS1" OR "PAD4" )
# NOT ( partial OR fragment OR hypothetical OR predicted OR DAP-seq OR
# ↪ "transcription factor" OR DNA-binding )
# '''

POSITIVE_QUERY = r'''

```

```

    ("Solanaceae"[Organism] NOT ("Solanum betaceum"[Organism] OR "Solanum_
    ↪melongena"[Organism] OR "Solanum tuberosum"[Organism] OR "Physalis_
    ↪peruviana"[Organism]))
    AND (TMV OR ToMV OR ToBRFV OR PMMoV OR TMGMV OR PVY OR PepYMV OR TEV OR PVA_
    ↪OR PVX OR TSWV OR GRSV OR TCSV OR CMV OR ToCV OR TICV OR AMV
    OR "virus resistance" OR "antiviral" OR "RNA silencing" OR "RDR" OR_
    ↪"Dicer-like" OR "Argonaute" OR "AGO" OR "eIF4E" OR "R gene" OR "Sw-5" OR_
    ↪"Tm-2" OR "RCY1"
    OR "Rx" OR "R protein" OR "NLR" OR "NBS-LRR" OR "NB-LRR" OR "TIR-NBS-LRR"_
    ↪OR "CC-NBS-LRR" OR "RLK" OR "RLP" OR "LRR receptor kinase"
    OR "leucine rich repeat receptor kinase" OR "plant disease resistance_
    ↪protein" OR "disease resistance protein" OR "RPM1" OR "RPS2" OR "RPS4" OR_
    ↪"RPS5"
    OR "RPP" OR "SNC1" OR "ADR1" OR "NDR1" OR "EDS1" OR "PAD4" OR "NB-ARC" OR_
    ↪"NBS-ARC" OR "TIR domain"
    OR "PRR" OR "LYK" OR "LRR" OR "LECRK" OR "TNL" OR "NB-LRR" OR "IL-1R"
    OR "NB-ARC domain" OR "CNL Domain" OR "NBS-ARC" OR "leucine-rich repeat_
    ↪domain" OR "TIR domain-containing" OR "LRR-containing" OR "coiled-coil"
    OR "leucine-rich repeat" OR "Kinase domain" OR "Nucleotide-binding site")
    NOT (partial OR fragment OR hypothetical OR predicted OR DAP-seq OR_
    ↪"transcription factor" OR DNA-binding OR microtom )
'''
# #OR microtom
# NEGATIVE_QUERY = r'''
# ( "Solanum lycopersicum"[Organism] OR "Nicotiana benthamiana"[Organism] OR_
# ↪"Capsicum annuum"[Organism] OR "Arabidopsis thaliana"[Organism])
# AND ( "actin" OR "tubulin" OR "ribosomal protein" OR "ATP synthase" OR_
# ↪"photosystem" OR "elongation factor" OR "glycolysis" OR "metabolic enzyme" )
# NOT ( "virus resistance" OR "antiviral" OR "RNA silencing" OR "RDR" OR_
# ↪"Dicer-like" OR "Argonaute" OR "AGO" OR "eIF4E" OR "R gene" OR "Sw-5" OR_
# ↪"Tm-2"
# OR "RCY1" OR "Rx" OR "R protein" OR "resistance" OR "NLR" OR "LRR" OR_
# ↪"immune" OR "virus" )
# '''
NEGATIVE_QUERY = r'''
    ("Solanaceae"[Organism] NOT ("Solanum melongena"[Organism] OR "Solanum_
    ↪tuberosum"[Organism] OR "Physalis peruviana"[Organism]))
    AND 100:3000[Sequence Length]
    NOT (TMV OR ToMV OR ToBRFV OR PMMoV OR TMGMV OR PVY OR PepYMV OR TEV OR PVA_
    ↪OR PVX OR TSWV OR GRSV OR TCSV OR CMV OR ToCV OR TICV OR AMV
    OR "virus resistance" OR "antiviral" OR "RNA silencing" OR "RDR" OR_
    ↪"Dicer-like" OR "Argonaute" OR "AGO" OR "eIF4E" OR "R gene" OR "Sw-5" OR_
    ↪"Tm-2" OR "RCY1"
    OR "Rx" OR "R protein" OR "NLR" OR "NBS-LRR" OR "NB-LRR" OR "TIR-NBS-LRR"_
    ↪OR "CC-NBS-LRR" OR "RLK" OR "RLP" OR "LRR receptor kinase"

```

```

    OR "leucine rich repeat receptor kinase" OR "plant disease resistance_
↳protein" OR "disease resistance protein" OR "RPM1" OR "RPS2" OR "RPS4" OR_
↳"RPS5"
    OR "PRR" OR "LYK" OR "LRR" OR "LECRK" OR "TNL" OR "NB-LRR" OR "IL-1R" OR_
↳"NB-ARC" OR "NB-ARC domain"
    OR "RPP" OR "SNC1" OR "ADR1" OR "NDR1" OR "EDS1" OR "PAD4" OR "coiled-coil"_
↳ OR "leucine-rich repeat" OR "Kinase domain" OR "Nucleotide-binding site")
    NOT (partial OR fragment OR hypothetical OR predicted OR uncharacterized OR_
↳DAP-seq OR "transcription factor" OR DNA-binding OR microtom )
'''

# Dados dos transcriptomas para a aplicação - previsão de genes de resistência
# ARABIDOPSIS_QUERY = '''
#     "Arabidopsis thaliana"[Organism] AND (transcriptome OR RNA-Seq OR TSA OR_
↳mRNA)
#     NOT (genome OR chromosome OR scaffold OR contig OR partial OR fragment_
↳OR microtom)
#     AND 1500:30000[Sequence Length]
#     '''
ARABIDOPSIS_QUERY = '''
    "Arabidopsis thaliana"[Organism] AND (biomol_mrna[PROP] OR tsa[keyword])
    AND 1500:30000[Sequence Length]
'''

SOLANUM_MELONGENA = '''
    "Solanum melongena"[Organism] AND (biomol_mrna[PROP] OR tsa[keyword])
    AND 1500:30000[Sequence Length]
'''

TUBEROSUM_QUERY = '''
    "Solanum tuberosum"[Organism] AND (biomol_mrna[PROP] OR tsa[keyword])
    AND 1500:30000[Sequence Length]
'''

PHYSALIS_QUERY = '''
    "Physalis peruviana"[Organism] AND (biomol_mrna[PROP] OR tsa[keyword])
    AND 1500:30000[Sequence Length]
'''

# =====
# DOWNLOAD NCBI
# =====

def baixar_proteinas_ncbi(query, output_fasta, max_records=35000):
    print("\n=====")
    print("DOWNLOAD NCBI: ",output_fasta)

```

```

print("=====")
handle = Entrez.esearch(
    db="protein",
    term=query,
    retmax=max_records
)
record = Entrez.read(handle)
ids = record["IdList"]
print(f"Proteínas encontradas: {len(ids)}")
if len(ids) == 0:
    return
handle = Entrez.efetch(
    db="protein",
    id=ids,
    rettype="fasta",
    retmode="text"
)
with open(output_fasta, "w") as f:
    f.write(handle.read())
print(f"Arquivo salvo: {output_fasta}")

def baixar_transcriptoma(output_fasta, query):
    print("\n=====")
    print("DOWNLOAD TRANSCRIPTOMA: ", output_fasta)
    print("=====")
    handle = Entrez.esearch(
        db="nucleotide",
        term=query,
        retmax=50000
    )
    record = Entrez.read(handle)
    ids = record["IdList"]
    print(f"Transcritos encontrados: {len(ids)}")
    handle = Entrez.efetch(
        db="nucleotide",
        id=ids,
        rettype="fasta",
        retmode="text"
    )
    with open(output_fasta, "w") as f:
        f.write(handle.read())
    print(f"Arquivo salvo: {output_fasta}")

#Dados de Entrada - baixar arquivos FASTA das sequências de proteínas
POSITIVE_FASTA = f"{OUTPUT_DIR}/positive.fasta"
NEGATIVE_FASTA = f"{OUTPUT_DIR}/negative.fasta"

```

```
print("\n===== FINALIZADO CONFIGURAÇÃO DE PARÂMETROS =====\n")
```

===== FINALIZADO CONFIGURAÇÃO DE PARÂMETROS =====

```
[3]: inicio = datetime.now()
print("\n\nIniciando download da classe positiva: ", inicio.strftime("%H:%M:%S"))
baixar_proteinas_ncbi(POSITIVE_QUERY, POSITIVE_FASTA)
fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

Iniciando download da classe positiva: 12:38:42

```
=====
DOWNLOAD NCBI: pipelineAB/positive.fasta
=====
Proteínas encontradas: 34655
Arquivo salvo: pipelineAB/positive.fasta
```

Finalizado em: 12:40:38

Tempo decorrido: 0:01:56.422742

```
[5]: inicio = datetime.now()
print("\n\nIniciando download da classe negativa: ", inicio.strftime("%H:%M:%S"))
baixar_proteinas_ncbi(NEGATIVE_QUERY, NEGATIVE_FASTA)
fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

print("\n\nArquivos salvos na pasta: ", OUTPUT_DIR)
print("===== FIM COLETA DE DADOS TREINO/VALIDAÇÃO =====")
```

Iniciando download da classe negativa: 13:06:02

```
=====
DOWNLOAD NCBI: pipelineAB/negative.fasta
=====
Proteínas encontradas: 35000
Arquivo salvo: pipelineAB/negative.fasta
```

Finalizado em: 13:07:41

Tempo decorrido: 0:01:39.019544

Arquivos salvos na pasta: pipelineAB

===== FIM COLETA DE DADOS TREINO/VALIDAÇÃO =====

```
[3]: #Dados para aplicação - baixar transcriptomas - RNA-Seq
transcriptoma_arabidopsis_fasta = (f"{OUTPUT_DIR}/arabidopsis_transcriptoma.
↳fasta")
transcriptoma_melongena_fasta = (f"{OUTPUT_DIR}/melongena_transcriptoma.fasta")
transcriptoma_tuberosum_fasta = (f"{OUTPUT_DIR}/tuberosum_transcriptoma.fasta")
transcriptoma_physalis_fasta = (f"{OUTPUT_DIR}/physalis_transcriptoma.fasta")
```

```
[7]: print("\n\nIniciando download de dados de transcriptmas")
      inicio = datetime.now()
      print("\n\nIniciando download ARABIDOPSIS: ", inicio.strftime("%H:%M:%S"))
      baixar_transcriptoma(transcriptoma_arabidopsis_fasta, ARABIDOPSIS_QUERY)
      fim = datetime.now()
      print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
      print("\nTempo decorrido:", fim - inicio)
```

Iniciando download de dados de transcriptmas

Iniciando download ARABIDOPSIS: 13:07:41

```
=====
DOWNLOAD TRANSCRIPTOMA: pipelineAB/arabidopsis_transcriptoma.fasta
=====
Transcritos encontrados: 49951
Arquivo salvo: pipelineAB/arabidopsis_transcriptoma.fasta
```

Finalizado em: 13:08:27

Tempo decorrido: 0:00:46.625619

```
[8]: inicio = datetime.now()
      print("\n\nIniciando download PHYSALIS: ", inicio.strftime("%H:%M:%S"))
      baixar_transcriptoma(transcriptoma_physalis_fasta, PHYSALIS_QUERY)
      fim = datetime.now()
      print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
      print("\nTempo decorrido:", fim - inicio)
```

Inciando download PHYSALIS: 13:08:27

```
=====
DOWNLOAD TRANSCRIPTOMA: pipelineAB/physalis_transcriptoma.fasta
=====
Transcritos encontrados: 21702
Arquivo salvo: pipelineAB/physalis_transcriptoma.fasta
```

Finalizado em: 13:09:14

Tempo decorrido: 0:00:47.051003

```
[9]: inicio = datetime.now()
print("\n\nInciando download MELONGENA: ", inicio.strftime("%H:%M:%S"))
baixar_transcriptoma(transcriptoma_melongena_fasta, SOLANUM_MELONGENA)
fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

Inciando download MELONGENA: 13:09:14

```
=====
DOWNLOAD TRANSCRIPTOMA: pipelineAB/melongena_transcriptoma.fasta
=====
Transcritos encontrados: 8555
Arquivo salvo: pipelineAB/melongena_transcriptoma.fasta
```

Finalizado em: 13:09:54

Tempo decorrido: 0:00:39.393130

```
[10]: inicio = datetime.now()
print("\n\nInciando download TUBEROSUM: ", inicio.strftime("%H:%M:%S"))
baixar_transcriptoma(transcriptoma_tuberosum_fasta, TUBEROSUM_QUERY)
fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

print("\n\nArquivos salvos na pasta: ", OUTPUT_DIR)
print("===== FIM COLETA DE DADOS APLICAÇÃO =====")
```


Iniciando download TUBEROSUM: 13:09:54

```
=====
DOWNLOAD TRANSCRIPTOMA: pipelineAB/tuberosum_transcriptoma.fasta
=====
Transcritos encontrados: 20832
Arquivo salvo: pipelineAB/tuberosum_transcriptoma.fasta
```

Finalizado em: 13:10:40

Tempo decorrido: 0:00:46.241459

Arquivos salvos na pasta: pipelineAB
===== FIM COLETA DE DADOS APLICAÇÃO =====

```
[4]: '''
    Tratamento dos dados
        - Criar Dataframes com os conjuntos de dados de treino e validação
        - Executar CD-Hit para eliminar sequencias muito semelhantes
        - Extração de atributos (features)

    '''

# =====
# AAC
# =====
# composição de aminoácidos - transformar a proteína em dados numéricos
# conta a frequência de cada aminoácido em uma sequência de proteína
# como são 20 aminoácidos possíveis, irá gerar 20 features com seus percentuais
# com essa frequência consegue ter um parâmetro para distinguir as proteínas
# por exemplo proteínas de resistência tem geralmente certa frequência de
    ↳ aminoácidos
# mas ignora a ordem - por isso a sequência de dipeptídeos irá incrementar o
    ↳ modelo considerando a ordem
# sequência de 20 aminoácidos
AMINO_ACIDOS = list("ACDEFGHIKLMNPQRSTVWY")

motifs = {
    # =====
    # NLR (CNL/TNL)
    # =====
    "P_LOOP": r"[AGST]{3,5}GK[TS]",
    "KINASE2": r"[ALIVMFYW]{3,5}DD[VILW][WFY]",
    "GLPL": r"G[LVI][PAST][LIVAF]",
```

```

    "MHD": r"[MVIL][HN][DE][VILAST]",
    "RNBS_A": r"F.DL.{0,3}AW",
    "RNBS_B": r"G[SA].{2,6}T",
    "RNBS_C": r"Y.[VIL].{1,3}[LS]",
    "RNBS_D": r"C.{2,8}P",

    # =====
    # Protein Kinase (KIN/RLK)
    # =====
    "HRD": r"HRD[LIVMFY]",
    "DFG": r"DFG",

    # =====
    # LRR (RLP/RLK)
    # =====
    "LRR1": r"L..L..L..N",
    "LRR2": r"L[A-Z]{2}L[A-Z]{2}L[A-Z]{2}[NCT]"
}

def calcular_aac(seq):
    seq = seq.upper()
    total = len(seq)
    if total == 0:
        return [0] * len(AMINO_ACIDOS)
    counts = Counter(seq)
    return [
        counts.get(aa, 0) / total
        for aa in AMINO_ACIDOS
    ]

# =====
# DIPEPTÍDEOS
# =====
'''
encontrar pares de aminoácidos consecutivos em uma sequência protéica □
↳-sequência de dois aminoácidos
considera a ordem dos aminoácidos
há certos padrões comuns para
para 20 aminoácidos, há possibilidade de 20*20 = 400 dipeptídeos - features
'''

DIPEPTIDES = [
    a + b
    for a in AMINO_ACIDOS
    for b in AMINO_ACIDOS
]

```

```

def calcular_dipeptideos(seq):
    seq = seq.upper()
    total = len(seq) - 1
    if total <= 0:
        return [0] * len(DIPEPTIDES)
    counts = Counter([
        seq[i:i+2]
        for i in range(total)
    ])
    return [
        counts.get(dp, 0) / total
        for dp in DIPEPTIDES
    ]

def localizar_motivos(seq):
    posicoes = {}
    for nome, pattern in motifs.items():
        match = re.search(pattern, seq)

        if match:
            posicoes[nome] = match.start()
        else:
            posicoes[nome] = None

    return posicoes

def calcular_distancias(posicoes):

    def distancia(a, b):
        if posicoes[a] is None or posicoes[b] is None:
            return -1
        return posicoes[b] - posicoes[a]

    return {
        "dist_ploop_kinase2": distancia("P_LOOP", "KINASE2"),
        "dist_kinase2_glpl": distancia("KINASE2", "GLPL"),
        "dist_glpl_mhd": distancia("GLPL", "MHD")
    }

def contar_motivos(posicoes):
    return sum(
        1 for v in posicoes.values()
        if v is not None
    )

```

```

def extrair_motifs(seq):

    return [
        int(bool(re.search(pattern, seq)))
        for pattern in motifs.values()
    ]

# =====
# FEATURES - atributos
# =====
#calculando features
def extrair_features_proteina(seq):
    encontrados = {}
    seq = re.sub(r"[^ACDEFGHIKLMNPQRSTVWY]", "", seq)
    if len(seq) < 50:
        return None
    features = []
    features.append(len(seq)) # 1 - comprimento da cadeia
    features.extend(calcular_aac(seq)) # 20 composição de aminoácidos -
    ↪ frequência de cada um na sequência
    features.extend(calcular_dipeptideos(seq)) #400 possibilidades - considera
    ↪ a ordem dos aminoácidos para buscar os mais frequentes em proteínas de
    ↪ resistência

    features.extend(extrair_motifs(seq))
    # posições dos motivos
    posicoes = localizar_motivos(seq)
    # número de motivos encontrados
    num_motifs_detectados = contar_motivos(posicoes)
    # distâncias entre motivos
    distancias = calcular_distancias(posicoes)

    features.append(num_motifs_detectados)
    protein_length = len(seq)

    features.append(distancias["dist_ploop_kinase2"])
    features.append(distancias["dist_kinase2_glp1"])
    features.append(distancias["dist_glp1_mhd"])

    features.append(
        distancias["dist_ploop_kinase2"]/protein_length
        if distancias["dist_ploop_kinase2"] != -1 else -1
    )
    features.append(
        distancias["dist_kinase2_glp1"]/protein_length

```

```

        if distancias["dist_kinase2_glpl"] != -1 else -1
    )
    features.append(
        distancias["dist_glpl_mhd"]/protein_length
        if distancias["dist_glpl_mhd"] != -1 else -1
    )

    features.append(int(posicoes["P_LOOP"] is not None))
    features.append(int(posicoes["KINASE2"] is not None))
    features.append(int(posicoes["GLPL"] is not None))
    features.append(int(posicoes["MHD"] is not None))

    for nome, regex in motivos.items():
        encontrados[nome] = int( re.search(regex, seq) is not None)

    # features.append(encontrados[nome])

    num_nlr = (
        encontrados["P_LOOP"] +
        encontrados["KINASE2"] +
        encontrados["GLPL"] +
        encontrados["MHD"]
    )

    num_kinase = (
        encontrados["HRD"] +
        encontrados["DFG"]
    )

    num_lrr = (
        encontrados["LRR1"] +
        encontrados["LRR2"]
    )

    features.extend([
        num_nlr,
        num_kinase,
        num_lrr
    ])

    return features

# =====
# DATASET
# =====
motif_cols = [
    "P_LOOP",

```

```

    "KINASE2",
    "GLPL",
    "MHD",
    "RNBS_A",
    "RNBS_B",
    "RNBS_C",
    "RNBS_D",
    "HRD",
    "DFG",
    "LRR1",
    "LRR2",
    "NUM_NLR_MOTIFS",
    "NUM_KINASE_MOTIFS",
    "NUM_LRR_MOTIFS"
]

def criar_dataset(positive_fasta, negative_fasta):
    data = []
    columns = (
        ["protein_length"]
        + [f"AAC_{aa}" for aa in AMINO_ACIDOS]
        + [f"DIPEP_{dp}" for dp in DIPEPTIDES]
        + motif_cols
        + ["num_motifs_detectados"]
        + ["dist_ploop_kinase2"]
        + ["dist_kinase2_glpl"]
        + ["dist_glpl_mhd"]
        + ["dist_ploop_kinase2_norm"]
        + ["dist_kinase2_glpl_norm"]
        + ["dist_glpl_mhd_norm"]
        + ["has_ploop"]
        + ["has_kinase2"]
        + ["has_glpl"]
        + ["has_mhd"]
    )

    # POSITIVOS
    for record in SeqIO.parse(positive_fasta, "fasta"):
        seq = str(record.seq)
        feats = extrair_features_proteina(seq)
        if feats is None:
            continue
        row = feats + [1, record.id]
        data.append(row)

    # NEGATIVOS
    for record in SeqIO.parse(negative_fasta, "fasta"):

```

```

        seq = str(record.seq)
        feats = extrair_features_proteina(seq)
        if feats is None:
            continue
        row = feats + [0, record.id]
        data.append(row)

df = pd.DataFrame(
    data,
    columns=columns + ["target", "id"]
)
print("\nTotal de Features/Atributos:", len(columns)) # quantidade
print("\nFeatures/atributos:")
print(columns)
return df, columns

# CD-Hit
# chama o programa CD-HIT do sistema operacional
def executar_cdhit(
    input_fasta,
    output_fasta,
    identity=0.7 #0.5
):
    cmd = [
        "cd-hit",
        "-i", input_fasta,
        "-o", output_fasta,
        "-c", str(identity),
        "-n", "5",
        "-M", "30000",
        "-T", "4"
    ]

    print("\nExecutando CD-HIT...")
    subprocess.run(cmd, check=True)
    print("CD-HIT concluído.")

def limpar(seq_id):
    seq_id = seq_id.strip()
    seq_id = seq_id.split()[0]
    return seq_id

def ler_clusters_cdhit(cluster_file):
    clusters = {}
    cluster_id = None
    with open(cluster_file) as f:
        for line in f:

```

```

        line = line.strip()
        if line.startswith(">Cluster"):
            cluster_id = line.replace(">Cluster ", "")
            clusters[cluster_id] = []
        else:
            seq_id = line.split(">")[1].split("...")[0]
            seq_id = limpar(seq_id)
            clusters[cluster_id].append(seq_id)
    return clusters

inicio = datetime.now()
print("\n\nIniciando Tratamento dos dados: ", inicio.strftime("%H:%M:%S"))

# executa o CD-Hit nos arquivos baixados
print("\n\nIniciar a execução do CD-HIT\n")
executar_cdhit(
    POSITIVE_FASTA,
    f"{OUTPUT_DIR}/positive_nr.fasta",
    identity=0.7
)

executar_cdhit(
    NEGATIVE_FASTA,
    f"{OUTPUT_DIR}/negative_nr.fasta",
    identity=0.7
)

# =====
# Montar os DATASETS
# =====

POSITIVE_FASTA = f"{OUTPUT_DIR}/positive_nr.fasta"
NEGATIVE_FASTA = f"{OUTPUT_DIR}/negative_nr.fasta"

df, columns = criar_dataset(
    POSITIVE_FASTA,
    NEGATIVE_FASTA
)

df["id"] = df["id"].apply(limpar)
# =====
# CLUSTERS
# =====

clusters_pos = ler_clusters_cdhit(
    f"{OUTPUT_DIR}/positive_nr.fasta.clstr"
)

```



```

clusters_neg = ler_clusters_cdhit(
    f"{OUTPUT_DIR}/negative_nr.fasta.clstr"
)

cluster_map = {}

for c, seqs in clusters_pos.items():
    for s in seqs:
        cluster_map[s] = f"POS_{c}"

for c, seqs in clusters_neg.items():
    for s in seqs:
        cluster_map[s] = f"NEG_{c}"

df["cluster"] = df["id"].map(cluster_map)

print("\n Clusters ausentes: ")
print(df["cluster"].isna().sum())
df = df.dropna(subset=["cluster"])

print("\nDataset original pós CD-HIT:", df.shape)

# =====
# NOVO BLOCO: BALANCEAMENTO IMPERATIVO (DOWNSAMPLING)
# =====
print("\n--- Executando Balanceamento de Classes ---")
df_positivos = df[df["target"] == 1]
df_negativos = df[df["target"] == 0]

print(f"Contagem inicial -> Positivos (R-genes): {len(df_positivos)} |  

    ↳Negativos (Background): {len(df_negativos)}")

# Se houver mais negativos do que positivos (cenário padrão), reduzimos os  

    ↳negativos
if len(df_negativos) > len(df_positivos):
    # O sample() escolhe aleatoriamente N linhas do grupo majoritário
    df_negativos_balanceados = df_negativos.sample(n=len(df_positivos),  

    ↳random_state=42)
    df = pd.concat([df_positivos, df_negativos_balanceados]).  

    ↳reset_index(drop=True)
    print(f"Subamostragem aplicada com sucesso nos Negativos.")
else:
    print("O dataset já está balanceado ou possui mais positivos que negativos,  

    ↳(incomum). Nenhuma ação foi tomada.")

```

```

print(f"Contagem final    -> Positivos: {len(df[df['target'] == 1])} | Negativos:
    ↳ {len(df[df['target'] == 0])}")
print("Dataset Final Pronto:", df.shape)

fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
print("\n\nFim tratamento dos dados...")

```

Iniciando Tratamento dos dados: 13:28:37

Iniciar a execução do CD-HIT

Executando CD-HIT...

```

=====
Program: CD-HIT, V4.8.1 (+OpenMP), Aug 20 2021, 08:39:56
Command: cd-hit -i pipelineAB/positive.fasta -o
         pipelineAB/positive_nr.fasta -c 0.7 -n 5 -M 30000 -T 4

```

Started: Thu Jun 11 13:28:37 2026

```

=====
                          Output
-----

```

```

total seq: 9999
longest and shortest : 3540 and 28
Total letters: 7105544
Sequences have been sorted

```

Approximated minimal memory consumption:

```

Sequence      : 8M
Buffer        : 4 X 11M = 45M
Table         : 2 X 65M = 131M
Miscellaneous  : 0M
Total         : 184M

```

Table limit with the given memory limit:

```

Max number of representatives: 4000000
Max number of word counting entries: 3726877013

```

```

# comparing sequences from      0 to      1666
.----- new table with      366 representatives
# comparing sequences from     1666 to     3054

```

```

----- 600 remaining sequences to the next cycle
----- new table with      186 representatives
# comparing sequences from    2454 to    3711
----- 757 remaining sequences to the next cycle
----- new table with      118 representatives
# comparing sequences from    2954 to    4128
----- 696 remaining sequences to the next cycle
----- new table with      101 representatives
# comparing sequences from    3432 to    4526
----- 620 remaining sequences to the next cycle
----- new table with      105 representatives
# comparing sequences from    3906 to    4921
----- 630 remaining sequences to the next cycle
----- new table with      100 representatives
# comparing sequences from    4291 to    5242
----- 535 remaining sequences to the next cycle
----- new table with      100 representatives
# comparing sequences from    4707 to    5589
----- 375 remaining sequences to the next cycle
----- new table with      100 representatives
# comparing sequences from    5214 to    6011
----- 184 remaining sequences to the next cycle
----- new table with      100 representatives
# comparing sequences from    5827 to    6522
----- 303 remaining sequences to the next cycle
----- new table with      100 representatives
# comparing sequences from    6219 to    6849
----- 297 remaining sequences to the next cycle
----- new table with      100 representatives
# comparing sequences from    6552 to    7126
----- 256 remaining sequences to the next cycle
----- new table with      100 representatives
# comparing sequences from    6870 to    7391
----- 218 remaining sequences to the next cycle
----- new table with      100 representatives
# comparing sequences from    7173 to    7644
----- 131 remaining sequences to the next cycle
----- new table with      100 representatives
# comparing sequences from    7513 to    7927
----- 30 remaining sequences to the next cycle
----- new table with      100 representatives
# comparing sequences from    7897 to    8247
...----- new table with      94 representatives
# comparing sequences from    8247 to    8539
...----- new table with      91 representatives
# comparing sequences from    8539 to    8782
...----- new table with      85 representatives
# comparing sequences from    8782 to    8984

```

```

99.9%----- new table with          5 representatives
# comparing sequences from      8984 to      9153
...----- new table with          50 representatives
# comparing sequences from      9153 to      9999
...----- new table with        179 representatives

```

9999 finished 2380 clusters

Approximated maximum memory consumption: 190M
 writing new database
 writing clustering information
 program completed !

Total CPU time 2.77
 CD-HIT concluído.

Executando CD-HIT...

```

=====
Program: CD-HIT, V4.8.1 (+OpenMP), Aug 20 2021, 08:39:56
Command: cd-hit -i pipelineAB/negative.fasta -o
         pipelineAB/negative_nr.fasta -c 0.7 -n 5 -M 30000 -T 4

```

Started: Thu Jun 11 13:28:39 2026

```

=====
                        Output
-----

```

```

total seq: 9999
longest and shortest : 2993 and 100
Total letters: 4226744
Sequences have been sorted

```

Approximated minimal memory consumption:

```

Sequence      : 5M
Buffer        : 4 X 11M = 44M
Table         : 2 X 65M = 131M
Miscellaneous  : 0M
Total         : 181M

```

Table limit with the given memory limit:

Max number of representatives: 4000000
 Max number of word counting entries: 3727296166

```

# comparing sequences from      0 to      1666
.----- new table with      1378 representatives
# comparing sequences from      1666 to      3054
----- 674 remaining sequences to the next cycle
----- new table with        590 representatives
# comparing sequences from      2380 to      3649

```

```

-----      803 remaining sequences to the next cycle
----- new table with      301 representatives
# comparing sequences from      2846 to      4038
-----      824 remaining sequences to the next cycle
----- new table with      248 representatives
# comparing sequences from      3214 to      4344
-----      789 remaining sequences to the next cycle
----- new table with      288 representatives
# comparing sequences from      3555 to      4629
-----      750 remaining sequences to the next cycle
----- new table with      272 representatives
# comparing sequences from      3879 to      4899
-----      696 remaining sequences to the next cycle
----- new table with      259 representatives
# comparing sequences from      4203 to      5169
-----      664 remaining sequences to the next cycle
----- new table with      221 representatives
# comparing sequences from      4505 to      5420
-----      639 remaining sequences to the next cycle
----- new table with      228 representatives
# comparing sequences from      4781 to      5650
-----      574 remaining sequences to the next cycle
----- new table with      191 representatives
# comparing sequences from      5076 to      5896
-----      550 remaining sequences to the next cycle
----- new table with      203 representatives
# comparing sequences from      5346 to      6121
-----      541 remaining sequences to the next cycle
----- new table with      188 representatives
# comparing sequences from      5580 to      6316
-----      507 remaining sequences to the next cycle
----- new table with      155 representatives
# comparing sequences from      5809 to      6507
-----      462 remaining sequences to the next cycle
----- new table with      179 representatives
# comparing sequences from      6045 to      6704
-----      440 remaining sequences to the next cycle
----- new table with      170 representatives
# comparing sequences from      6264 to      6886
-----      425 remaining sequences to the next cycle
----- new table with      130 representatives
# comparing sequences from      6461 to      7050
-----      392 remaining sequences to the next cycle
----- new table with      132 representatives
# comparing sequences from      6658 to      7214
-----      380 remaining sequences to the next cycle
----- new table with      137 representatives
# comparing sequences from      6834 to      7361

```

```

----- 336 remaining sequences to the next cycle
----- new table with 120 representatives
# comparing sequences from 7025 to 7520
----- 317 remaining sequences to the next cycle
----- new table with 126 representatives
# comparing sequences from 7203 to 7669
----- 285 remaining sequences to the next cycle
----- new table with 138 representatives
# comparing sequences from 7384 to 7819
----- 272 remaining sequences to the next cycle
----- new table with 101 representatives
# comparing sequences from 7547 to 7955
----- 257 remaining sequences to the next cycle
----- new table with 119 representatives
# comparing sequences from 7698 to 8081
----- 217 remaining sequences to the next cycle
----- new table with 100 representatives
# comparing sequences from 7864 to 8219
----- 225 remaining sequences to the next cycle
----- new table with 100 representatives
# comparing sequences from 7994 to 8328
----- 186 remaining sequences to the next cycle
----- new table with 100 representatives
# comparing sequences from 8142 to 8451
----- 163 remaining sequences to the next cycle
----- new table with 100 representatives
# comparing sequences from 8288 to 8573
----- 133 remaining sequences to the next cycle
----- new table with 100 representatives
# comparing sequences from 8440 to 8699
----- 77 remaining sequences to the next cycle
----- new table with 100 representatives
# comparing sequences from 8622 to 8851
----- 40 remaining sequences to the next cycle
----- new table with 100 representatives
# comparing sequences from 8811 to 9009
----- 42 remaining sequences to the next cycle
----- new table with 100 representatives
# comparing sequences from 8967 to 9139
----- 26 remaining sequences to the next cycle
----- new table with 100 representatives
# comparing sequences from 9113 to 9999
...----- new table with 410 representatives

```

9999 finished 7184 clusters

Approximated maximum memory consumption: 194M
writing new database

writing clustering information
program completed !

Total CPU time 1.69
CD-HIT concluído.

Total de Features/Atributos: 447

Features/atributos:

['protein_length', 'AAC_A', 'AAC_C', 'AAC_D', 'AAC_E', 'AAC_F', 'AAC_G',
'AAC_H', 'AAC_I', 'AAC_K', 'AAC_L', 'AAC_M', 'AAC_N', 'AAC_P', 'AAC_Q', 'AAC_R',
'AAC_S', 'AAC_T', 'AAC_V', 'AAC_W', 'AAC_Y', 'DIPEP_AA', 'DIPEP_AC', 'DIPEP_AD',
'DIPEP_AE', 'DIPEP_AF', 'DIPEP_AG', 'DIPEP_AH', 'DIPEP_AI', 'DIPEP_AK',
'DIPEP_AL', 'DIPEP_AM', 'DIPEP_AN', 'DIPEP_AP', 'DIPEP_AQ', 'DIPEP_AR',
'DIPEP_AS', 'DIPEP_AT', 'DIPEP_AV', 'DIPEP_AW', 'DIPEP_AY', 'DIPEP_CA',
'DIPEP_CC', 'DIPEP_CD', 'DIPEP_CE', 'DIPEP_CF', 'DIPEP_CG', 'DIPEP_CH',
'DIPEP_CI', 'DIPEP_CK', 'DIPEP_CL', 'DIPEP_CM', 'DIPEP_CN', 'DIPEP_CP',
'DIPEP_CQ', 'DIPEP_CR', 'DIPEP_CS', 'DIPEP_CT', 'DIPEP_CV', 'DIPEP_CW',
'DIPEP_CY', 'DIPEP_DA', 'DIPEP_DC', 'DIPEP_DD', 'DIPEP_DE', 'DIPEP_DF',
'DIPEP_DG', 'DIPEP_DH', 'DIPEP_DI', 'DIPEP_DK', 'DIPEP_DL', 'DIPEP_DM',
'DIPEP_DN', 'DIPEP_DP', 'DIPEP_DQ', 'DIPEP_DR', 'DIPEP_DS', 'DIPEP_DT',
'DIPEP_DV', 'DIPEP_DW', 'DIPEP_DY', 'DIPEP_EA', 'DIPEP_EC', 'DIPEP_ED',
'DIPEP_EE', 'DIPEP_EF', 'DIPEP_EG', 'DIPEP_EH', 'DIPEP_EI', 'DIPEP_EK',
'DIPEP_EL', 'DIPEP_EM', 'DIPEP_EN', 'DIPEP_EP', 'DIPEP_EQ', 'DIPEP_ER',
'DIPEP_ES', 'DIPEP_ET', 'DIPEP_EV', 'DIPEP_EW', 'DIPEP_EY', 'DIPEP_FA',
'DIPEP_FC', 'DIPEP_FD', 'DIPEP_FE', 'DIPEP_FF', 'DIPEP_FG', 'DIPEP_FH',
'DIPEP_FI', 'DIPEP_FK', 'DIPEP_FL', 'DIPEP_FM', 'DIPEP_FN', 'DIPEP_FP',
'DIPEP_FQ', 'DIPEP_FR', 'DIPEP_FS', 'DIPEP_FT', 'DIPEP_FV', 'DIPEP_FW',
'DIPEP_FY', 'DIPEP_GA', 'DIPEP_GC', 'DIPEP_GD', 'DIPEP_GE', 'DIPEP_GF',
'DIPEP_GG', 'DIPEP_GH', 'DIPEP_GI', 'DIPEP_GK', 'DIPEP_GL', 'DIPEP_GM',
'DIPEP_GN', 'DIPEP_GP', 'DIPEP_GQ', 'DIPEP_GR', 'DIPEP_GS', 'DIPEP_GT',
'DIPEP_GV', 'DIPEP_GW', 'DIPEP_GY', 'DIPEP_HA', 'DIPEP_HC', 'DIPEP_HD',
'DIPEP_HE', 'DIPEP_HF', 'DIPEP_HG', 'DIPEP_HH', 'DIPEP_HI', 'DIPEP_HK',
'DIPEP_HL', 'DIPEP_HM', 'DIPEP_HN', 'DIPEP_HP', 'DIPEP_HQ', 'DIPEP_HR',
'DIPEP_HS', 'DIPEP_HT', 'DIPEP_HV', 'DIPEP_HW', 'DIPEP_HY', 'DIPEP_IA',
'DIPEP_IC', 'DIPEP_ID', 'DIPEP_IE', 'DIPEP_IF', 'DIPEP_IG', 'DIPEP_IH',
'DIPEP_II', 'DIPEP_IK', 'DIPEP_IL', 'DIPEP_IM', 'DIPEP_IN', 'DIPEP_IP',
'DIPEP_IQ', 'DIPEP_IR', 'DIPEP_IS', 'DIPEP_IT', 'DIPEP_IV', 'DIPEP_IW',
'DIPEP_IY', 'DIPEP_KA', 'DIPEP_KC', 'DIPEP_KD', 'DIPEP_KE', 'DIPEP_KF',
'DIPEP_KG', 'DIPEP_KH', 'DIPEP_KI', 'DIPEP_KK', 'DIPEP_KL', 'DIPEP_KM',
'DIPEP_KN', 'DIPEP_KP', 'DIPEP_KQ', 'DIPEP_KR', 'DIPEP_KS', 'DIPEP_KT',
'DIPEP_KV', 'DIPEP_KW', 'DIPEP_KY', 'DIPEP_LA', 'DIPEP_LC', 'DIPEP_LD',
'DIPEP_LE', 'DIPEP_LF', 'DIPEP_LG', 'DIPEP_LH', 'DIPEP_LI', 'DIPEP_LK',
'DIPEP_LL', 'DIPEP_LM', 'DIPEP_LN', 'DIPEP_LP', 'DIPEP_LQ', 'DIPEP_LR',
'DIPEP_LS', 'DIPEP_LT', 'DIPEP_LV', 'DIPEP_LW', 'DIPEP_LY', 'DIPEP_MA',
'DIPEP_MC', 'DIPEP_MD', 'DIPEP_ME', 'DIPEP_MF', 'DIPEP_MG', 'DIPEP_MH',
'DIPEP_MI', 'DIPEP_MK', 'DIPEP_ML', 'DIPEP_MM', 'DIPEP_MN', 'DIPEP_MP',
'DIPEP_MQ', 'DIPEP_MR', 'DIPEP_MS', 'DIPEP_MT', 'DIPEP_MV', 'DIPEP_MW',

'DIPEP_MY', 'DIPEP_NA', 'DIPEP_NC', 'DIPEP_ND', 'DIPEP_NE', 'DIPEP_NF',
 'DIPEP_NG', 'DIPEP_NH', 'DIPEP_NI', 'DIPEP_NK', 'DIPEP_NL', 'DIPEP_NM',
 'DIPEP_NN', 'DIPEP_NP', 'DIPEP_NQ', 'DIPEP_NR', 'DIPEP_NS', 'DIPEP_NT',
 'DIPEP_NV', 'DIPEP_NW', 'DIPEP_NY', 'DIPEP_PA', 'DIPEP_PC', 'DIPEP_PD',
 'DIPEP_PE', 'DIPEP_PF', 'DIPEP_PG', 'DIPEP_PH', 'DIPEP_PI', 'DIPEP_PK',
 'DIPEP_PL', 'DIPEP_PM', 'DIPEP_PN', 'DIPEP_PP', 'DIPEP_PQ', 'DIPEP_PR',
 'DIPEP_PS', 'DIPEP_PT', 'DIPEP_PV', 'DIPEP_PW', 'DIPEP_PY', 'DIPEP_QA',
 'DIPEP_QC', 'DIPEP_QD', 'DIPEP_QE', 'DIPEP_QF', 'DIPEP_QG', 'DIPEP_QH',
 'DIPEP_QI', 'DIPEP_QK', 'DIPEP_QL', 'DIPEP_QM', 'DIPEP_QN', 'DIPEP_QP',
 'DIPEP_QQ', 'DIPEP_QR', 'DIPEP_QS', 'DIPEP_QT', 'DIPEP_QV', 'DIPEP_QW',
 'DIPEP_QY', 'DIPEP_RA', 'DIPEP_RC', 'DIPEP_RD', 'DIPEP_RE', 'DIPEP_RF',
 'DIPEP_RG', 'DIPEP_RH', 'DIPEP_RI', 'DIPEP_RK', 'DIPEP_RL', 'DIPEP_RM',
 'DIPEP_RN', 'DIPEP_RP', 'DIPEP_RQ', 'DIPEP_RR', 'DIPEP_RS', 'DIPEP_RT',
 'DIPEP_RV', 'DIPEP_RW', 'DIPEP_RY', 'DIPEP_SA', 'DIPEP_SC', 'DIPEP_SD',
 'DIPEP_SE', 'DIPEP_SF', 'DIPEP_SG', 'DIPEP_SH', 'DIPEP_SI', 'DIPEP_SK',
 'DIPEP_SL', 'DIPEP_SM', 'DIPEP_SN', 'DIPEP_SP', 'DIPEP_SQ', 'DIPEP_SR',
 'DIPEP_SS', 'DIPEP_ST', 'DIPEP_SV', 'DIPEP_SW', 'DIPEP_SY', 'DIPEP_TA',
 'DIPEP_TC', 'DIPEP_TD', 'DIPEP_TE', 'DIPEP_TF', 'DIPEP_TG', 'DIPEP_TH',
 'DIPEP_TI', 'DIPEP_TK', 'DIPEP_TL', 'DIPEP_TM', 'DIPEP_TN', 'DIPEP_TP',
 'DIPEP_TQ', 'DIPEP_TR', 'DIPEP_TS', 'DIPEP_TT', 'DIPEP_TV', 'DIPEP_TW',
 'DIPEP_TY', 'DIPEP_VA', 'DIPEP_VC', 'DIPEP_VD', 'DIPEP_VE', 'DIPEP_VF',
 'DIPEP_VG', 'DIPEP_VH', 'DIPEP_VI', 'DIPEP_VK', 'DIPEP_VL', 'DIPEP_VM',
 'DIPEP_VN', 'DIPEP_VP', 'DIPEP_VQ', 'DIPEP_VR', 'DIPEP_VS', 'DIPEP_VT',
 'DIPEP_VV', 'DIPEP_VW', 'DIPEP_VY', 'DIPEP_WA', 'DIPEP_WC', 'DIPEP_WD',
 'DIPEP_WE', 'DIPEP_WF', 'DIPEP_WG', 'DIPEP_WH', 'DIPEP_WI', 'DIPEP_WK',
 'DIPEP_WL', 'DIPEP_WM', 'DIPEP_WN', 'DIPEP_WP', 'DIPEP_WQ', 'DIPEP_WR',
 'DIPEP_WS', 'DIPEP_WT', 'DIPEP_WV', 'DIPEP_WW', 'DIPEP_WY', 'DIPEP_YA',
 'DIPEP_YC', 'DIPEP_YD', 'DIPEP_YE', 'DIPEP_YF', 'DIPEP_YG', 'DIPEP_YH',
 'DIPEP_YI', 'DIPEP_YK', 'DIPEP_YL', 'DIPEP_YM', 'DIPEP_YN', 'DIPEP_YP',
 'DIPEP_YQ', 'DIPEP_YR', 'DIPEP_YS', 'DIPEP_YT', 'DIPEP_YV', 'DIPEP_YW',
 'DIPEP_YY', 'P_LOOP', 'KINASE2', 'GLPL', 'MHD', 'RNBS_A', 'RNBS_B', 'RNBS_C',
 'RNBS_D', 'HRD', 'DFG', 'LRR1', 'LRR2', 'NUM_NLR_MOTIFS', 'NUM_KINASE_MOTIFS',
 'NUM_LRR_MOTIFS', 'num_motifs_detectados', 'dist_ploop_kinase2',
 'dist_kinase2_glpl', 'dist_glpl_mhd', 'dist_ploop_kinase2_norm',
 'dist_kinase2_glpl_norm', 'dist_glpl_mhd_norm', 'has_ploop', 'has_kinase2',
 'has_glpl', 'has_mhd']

Clusters ausentes:

131

Dataset original pós CD-HIT: (9432, 450)

--- Executando Balanceamento de Classes ---

Contagem inicial -> Positivos (R-genes): 2336 | Negativos (Background): 7096

Subamostragem aplicada com sucesso nos Negativos.

Contagem final -> Positivos: 2336 | Negativos: 2336

Dataset Final Pronto: (4672, 450)

Finalizado em: 13:28:41

Tempo decorrido: 0:00:03.650458

Fim tratamento dos dados...

```
[5]: '''  
      k-Fold-Cross-Validation  
      '''  
  
def validacao_cruzada(nome, model, X_train, y_train, groups_train):  
  
    cv = StratifiedGroupKFold(  
        n_splits=5, #10  
        shuffle=True,  
        random_state=42  
    )  
    mcc_scorer = make_scorer(matthews_corrcoef)  
    scoring_dict = {  
        'recall': 'recall',  
        'roc_auc': 'roc_auc',  
        'mcc': mcc_scorer,  
        'accuracy': 'accuracy',  
        'precision': 'precision',  
        'f1': 'f1'  
    }  
    scores = cross_validate(  
        model,  
        X_train,  
        y_train,  
        groups=groups_train,  
        cv=cv,  
        scoring=scoring_dict,  
        n_jobs=-1  
    )  
    print(f"Finalizada a validação cruzada para: {nome}")  
    return scores
```

```
[10]: '''  
      Criação dos modelos/ Treinamento  
      '''  
  
# para melhor separar os conjuntos de treino e teste considerando os  
→ agrupamentos feitos no CD-HIT para sequências muito parecidas
```

```

def split_por_homologia(df):
    print(df.head(30))
    groups = df["cluster"]
    splitter = GroupShuffleSplit(
        n_splits=1,
        test_size=0.2,
        random_state=42
    )
    # cv = StratifiedGroupKFold(
    #     n_splits = 10,
    #     shuffle=True,
    #     random_state = 42
    # )

    train_idx, test_idx = next(
        splitter.split(df, groups=groups)
        # cv.split(df, df["target"], groups)
    )
    train_df = df.iloc[train_idx]
    test_df = df.iloc[test_idx]
    return train_df, test_df

def calcular_e_salvar_shap(nome, model, X_test, feature_names, output_dir):
    """
    Calcula os SHAP values para modelos de árvore e salva os gráficos
    ↳ explicativos.
    """
    print(f"\n[SHAP] Inicializando análise de interpretabilidade para: {nome}...
    ↳ ")

    # 1. Inicializa o explicador otimizado para árvores
    explainer = shap.TreeExplainer(model)

    # 2. Calcula os SHAP values para o conjunto de teste
    shap_values = explainer.shap_values(X_test)

    # 3. Tratamento de formato (Scikit-Learn Random Forest vs XGBoost)
    # O Random Forest do sklearn retorna uma lista contendo [valores_classe_0,
    ↳ valores_classe_1]
    # O XGBoost retorna diretamente a matriz de SHAP values para a classe alvo
    if isinstance(shap_values, list):
        # Selecionamos o índice 1 (classe positiva: proteínas de resistência/
        ↳ alvos)
        shap_values_pos = shap_values[1]
    elif isinstance(shap_values, np.ndarray) and len(shap_values.shape) == 3:
        # Tratamento para versões específicas do SHAP que geram arrays 3D
        shap_values_pos = shap_values[:, :, 1]

```

```

else:
    shap_values_pos = shap_values

    # 4. Gráfico 1: Summary Plot (Beeswarm)
    # Mostra o impacto biológico de cada feature (ex: se alta frequência
    ↪aumenta ou diminui a predição)
    plt.figure(figsize=(12, 8))
    shap.summary_plot(
        shap_values_pos,
        X_test,
        feature_names=feature_names,
        max_display=20, # Foco nas top 20 características
        show=False
    )
    # plt.title(f"SHAP Summary Plot (Top 20) - {nome}", fontsize=14, pad=20)
    plt.tight_layout()
    plt.savefig(f"{output_dir}/{nome}_shap_summary.png", bbox_inches="tight",
    ↪dpi=300)
    plt.close()

    # 5. Gráfico 2: Bar Plot (Importância Global)
    # Mostra a magnitude média do impacto de cada feature de forma absoluta
    plt.figure(figsize=(12, 8))
    shap.summary_plot(
        shap_values_pos,
        X_test,
        feature_names=feature_names,
        plot_type="bar",
        max_display=20,
        show=False
    )
    # plt.title(f"SHAP Global Feature Importance - {nome}", fontsize=14, pad=20)
    plt.tight_layout()
    plt.savefig(f"{output_dir}/{nome}_shap_bar.png", bbox_inches="tight",
    ↪dpi=300)
    plt.close()

    print(f"[SHAP] Gráficos salvos com sucesso em '{output_dir}/'!")

def calcular_shap_svm(nome, model, feature_names, X_train, X_test, y_test,
    ↪output_dir):
    # =====
    # IMPORTÂNCIA POR PERMUTAÇÃO PARA O SVM (Novo)
    # =====
    print("\n[SVM] Calculando a importância por permutação dos atributos...")

```

```

# Calcula a permutação no X_test (pode usar X_train se quiser focar no
↪ ajuste ao treino)
result_svm = permutation_importance(model, X_test, y_test, n_repeats=5,
↪ random_state=42, n_jobs=-1)

# Monta o DataFrame com os resultados
importancia_svm = pd.DataFrame({
    'feature': columns,
    'importance_mean': result_svm.importances_mean
}).sort_values('importance_mean', ascending=False)

print("\nTop 20 features mais importantes para o SVM:")
print(importancia_svm.head(20))

# Gera o gráfico de barras igual ao do XGBoost
top_svm = importancia_svm.head(20)
plt.figure(figsize=(10,8))
plt.barh(top_svm["feature"][::-1], top_svm["importance_mean"][::-1],
↪ color='seagreen')
plt.xlabel("Queda no Desempenho (Média)")
plt.ylabel("Feature / Atributo")
# plt.title("Top 20 Features - SVM Permutation Importance")
plt.tight_layout()
plt.savefig(f"{output_dir}/svm_importancia_permutacao.png",
↪ bbox_inches="tight", dpi=300)
plt.close()

idx_back = np.random.choice(
    len(X_train),
    min(100, len(X_train)),
    replace = False
)
back = X_train[idx_back]

idx_test = np.random.choice(
    len(X_test),
    min(200, len(X_test)),
    replace = False
)
x_test_sample = X_test[idx_test]

print('\nCriando explainer shap...\n')
explainer = shap.Explainer(model.predict, back)
shap_values = explainer(x_test_sample, max_evals=1000)
print("\nShap SHAP:", shap_values.values.shape)

plt.figure(figsize=(12, 8))

```

```

shap.summary_plot(
    shap_values.values,
    x_test_sample,
    feature_names=feature_names,
    # plot_type="bar",
    max_display=20,
    show=False
)
# plt.title(f"SHAP Global Feature Importance - {nome}", fontsize=14, pad=20)
plt.tight_layout()
plt.savefig(f"{output_dir}/{nome}_shap_bar_summary.png",
    ↪bbox_inches="tight", dpi=300)
plt.close()

print(f"[SVM] Gráfico salvo com sucesso em '{output_dir}/"
    ↪svm_importancia_permutacao.png'!")

# =====
# TREINAMENTO - treina os modelos
# =====

def avaliar_modelo(nome, model, X_train, y_train, X_test, y_test):
    print("\n=====")
    print(nome)
    print("=====")
    model.fit(X_train, y_train)

    probs = model.predict_proba(X_test)[: ,1]
    preds = (probs >= 0.5).astype(int)
    print(classification_report(y_test, preds))
    roc = roc_auc_score(y_test, probs)
    pr = average_precision_score(y_test, probs)
    mcc = matthews_corrcoef(y_test, preds)
    f1 = f1_score(y_test, preds)
    precision = precision_score(y_test, preds)
    recall = recall_score(y_test, preds)
    bal_acc = balanced_accuracy_score(y_test, preds)
    acc = accuracy_score(y_test, preds)

    print("ROC-AUC          :", roc)
    print("PR-AUC           :", pr)
    print("MCC              :", mcc)
    print("F1               :", f1)
    print("Precision        :", precision)
    print("Recall           :", recall)
    print("Balanced Accuracy:", bal_acc)
    print("Standard Accuracy:", acc)

```

```

# =====
# MATRIZ DE CONFUSÃO
# =====
cm = confusion_matrix(y_test, preds)
plt.figure(figsize=(5,5))
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Negativo", "Positivo"]
)
disp.plot(cmap="Blues")
# plt.title(f"Matriz de Confusão - {nome}")
plt.savefig(
    f"{OUTPUT_DIR}/{nome}_matriz_confusao.png",
    bbox_inches="tight"
)
plt.close()
# =====
# CURVA ROC
# =====
fpr, tpr, _ = roc_curve(y_test, probs)
roc_auc = auc(fpr, tpr)
plt.figure(figsize=(6,6))
plt.plot(
    fpr,
    tpr,
    label=f"AUC = {roc_auc:.4f}"
)
plt.plot([0,1], [0,1], linestyle="--")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
# plt.title(f"Curva ROC - {nome}")
plt.legend(loc="lower right")
plt.savefig(
    f"{OUTPUT_DIR}/{nome}_roc.png",
    bbox_inches="tight"
)
plt.close()

return {
    "Modelo": nome,
    "ROC_AUC": roc,
    "F1": f1,
    "Precision": precision,
    "Recall": recall,
    "Balanced_Accuracy": bal_acc,
    "Accuracy": acc,
}

```

```

        "MCC":mcc
    }, model

# =====
# Separação Treino/Teste 80/20
# SPLIT POR HOMOLOGIA
# =====
inicio_treinamento = datetime.now()
print("\n\nIniciando Treinamento dos modelos: ", inicio_treinamento.
      ↪strftime("%H:%M:%S"))

train_df, test_df = split_por_homologia(df)
X_train = train_df[columns]
y_train = train_df["target"]
X_test = test_df[columns]
y_test = test_df["target"]
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# =====
# MODELOS
# =====

rf = RandomForestClassifier(
    n_estimators=1000,
    max_depth = 12,
    min_samples_split=4,
    max_features='sqrt',
    random_state=42,
    class_weight="balanced",
    n_jobs = -1
)
xgb = XGBClassifier(
    n_estimators=1200,
    max_depth=5,
    learning_rate=0.01,
    subsample=0.8,
    colsample_bytree=0.8,
    eval_metric="logloss",
    random_state=42,
    min_child_weight=3,
    reg_alpha=0.5,
    reg_lambda=2,
    scale_pos_weight=1.2
)

```

```

svm = SVC(
    probability=True,
    kernel="rbf",
    class_weight="balanced",
    random_state=42,
    C=2,
    gamma='scale'
)
scores = []
groups_train = train_df["cluster"]

print("\n\n Inciando validação cruzada");
scores_rf = validacao_cruzada('RF',rf,X_train, y_train, groups_train)
scores_xgb = validacao_cruzada('XGB',xgb, X_train, y_train, groups_train)
scores_svm = validacao_cruzada('SVM',svm, X_train, y_train, groups_train)

print("Cross-Validation:RF")
for k,v in scores_rf.items():
    if "test" in k:
        print(f"{k}: {v.mean():.4f} ± {v.std():.4f}")

print("Cross-Validation XGB")
for k,v in scores_xgb.items():
    if "test" in k:
        print(f"{k}: {v.mean():.4f} ± {v.std():.4f}")

print("Cross-Validation SVM")
for k,v in scores_svm.items():
    if "test" in k:
        print(f"{k}: {v.mean():.4f} ± {v.std():.4f}")

resultados_cv = pd.DataFrame({
    'Modelo': ['RF', 'XGB', 'SVM'],
    'Accuracy': [
        scores_rf['test_accuracy'].mean(),
        scores_xgb['test_accuracy'].mean(),
        scores_svm['test_accuracy'].mean(),
    ],
    'Recall': [
        scores_rf['test_recall'].mean(),
        scores_xgb['test_recall'].mean(),
        scores_svm['test_recall'].mean(),
    ],
    'Precision': [
        scores_rf['test_precision'].mean(),
        scores_xgb['test_precision'].mean(),
    ]
})

```



```

        scores_svm['test_precision'].mean(),
    ],
    'F1': [
        scores_rf['test_f1'].mean(),
        scores_xgb['test_f1'].mean(),
        scores_svm['test_f1'].mean(),
    ],
    'ROC_AUC': [
        scores_rf['test_roc_auc'].mean(),
        scores_xgb['test_roc_auc'].mean(),
        scores_svm['test_roc_auc'].mean(),
    ],
    'MCC': [
        scores_rf['test_mcc'].mean(),
        scores_xgb['test_mcc'].mean(),
        scores_svm['test_mcc'].mean(),
    ]
])
print(resultados_cv)
resultados_cv.to_csv(
    f"{OUTPUT_DIR}/metricas_cv.csv",
    index=False
)
dados = []

for score in scores_rf['test_roc_auc']:
    dados.append(['RF', score])
for score in scores_xgb['test_roc_auc']:
    dados.append(['XGB', score])
for score in scores_svm['test_roc_auc']:
    dados.append(['SVM', score])

grafico = pd.DataFrame(dados, columns=['Modelo', 'ROC_AUC'])
grafico.boxplot(by='Modelo')
plt.ylabel("ROC-AUC")
# plt.title("10-Fold Cross Validation")
plt.suptitle("")
plt.tight_layout()
# plt.show()
plt.savefig(
    f"{OUTPUT_DIR}/kfold_cross_validation_roc.png",
    bbox_inches="tight"
)
plt.close()

```

```
#####
# Treinamento
inicio = datetime.now()
print("\n\nIniciando treinamento RF: ", inicio.strftime("%H:%M:%S"))
results = []
r1, rf_model = avaliar_modelo(
    "Random Forest",
    rf,
    X_train,
    y_train,
    X_test,
    y_test
)
results.append(r1)
fim = datetime.now()
print("\n\nFinalizado treinamento RF em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

calcular_e_salvar_shap("Random Forest", rf_model, X_test, columns, OUTPUT_DIR )

inicio = datetime.now()
print("\n\nIniciando treinamento XGBoost: ", inicio.strftime("%H:%M:%S"))
r2, xgb_model = avaliar_modelo(
    "XGBoost",
    xgb,
    X_train,
    y_train,
    X_test,
    y_test
)
results.append(r2)
fim = datetime.now()
print("\n\nFinalizado em XGBoost:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

calcular_e_salvar_shap("XGBoost", xgb_model, X_test, columns, OUTPUT_DIR )

inicio = datetime.now()
print("\n\nIniciando treinamento SVM: ", inicio.strftime("%H:%M:%S"))
r3, svm_model = avaliar_modelo(
    "SVM",
    svm,
    X_train,
    y_train,
    X_test,
    y_test
)

```

```

results.append(r3)
fim = datetime.now()
print("\n\nFinalizado em SVM:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

calcular_shap_svm("SVM", svm_model, columns, X_train, X_test, y_test, OUTPUT_DIR)

fim = datetime.now()
print("\n\nFinalizado fase de treinamento em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio_treinamento)

print("\nFim criação/treino dos modelos\n")

```

Iniciando Treinamento dos modelos: 15:20:53

	protein_length	AAC_A	AAC_C	AAC_D	AAC_E	AAC_F	\
0	231	0.090909	0.012987	0.069264	0.077922	0.051948	
1	638	0.061129	0.012539	0.047022	0.053292	0.040752	
2	1249	0.032826	0.013611	0.078463	0.082466	0.044836	
3	201	0.039801	0.004975	0.034826	0.044776	0.019900	
4	670	0.049254	0.016418	0.052239	0.062687	0.046269	
5	674	0.060831	0.011869	0.051929	0.048961	0.044510	
6	888	0.055180	0.018018	0.061937	0.072072	0.045045	
7	662	0.069486	0.012085	0.052870	0.067976	0.051360	
8	385	0.023377	0.023377	0.044156	0.038961	0.054545	
9	504	0.103175	0.019841	0.045635	0.071429	0.029762	
10	704	0.049716	0.007102	0.080966	0.117898	0.041193	
11	535	0.095327	0.009346	0.020561	0.026168	0.082243	
12	677	0.051699	0.023634	0.053176	0.062038	0.051699	
13	721	0.058252	0.015257	0.052705	0.055479	0.054092	
14	450	0.040000	0.028889	0.035556	0.082222	0.024444	
15	294	0.078231	0.017007	0.040816	0.068027	0.027211	
16	1251	0.035971	0.031974	0.053557	0.067946	0.041567	
17	283	0.045936	0.035336	0.060071	0.063604	0.053004	
18	2293	0.037069	0.023550	0.056258	0.067161	0.038814	
19	813	0.027060	0.027060	0.027060	0.045510	0.049200	
20	1194	0.040201	0.018425	0.069514	0.069514	0.049414	
21	1028	0.050584	0.024319	0.050584	0.040856	0.055447	
22	651	0.046083	0.058372	0.064516	0.041475	0.029186	
23	808	0.027228	0.023515	0.043317	0.044554	0.045792	
24	906	0.030905	0.016556	0.046358	0.051876	0.059603	
25	128	0.054688	0.007812	0.046875	0.203125	0.031250	
26	1222	0.108838	0.010638	0.046645	0.084288	0.034370	
27	261	0.026820	0.019157	0.038314	0.088123	0.038314	
28	209	0.009569	0.019139	0.057416	0.038278	0.066986	
29	227	0.039648	0.044053	0.101322	0.079295	0.035242	

	AAC_G	AAC_H	AAC_I	AAC_K	...	dist_ploop_kinase2_norm	\
0	0.077922	0.038961	0.043290	0.082251	...	1	
1	0.062696	0.028213	0.062696	0.073668	...	0	
2	0.040032	0.029624	0.064051	0.077662	...	1	
3	0.054726	0.024876	0.074627	0.084577	...	0	
4	0.073134	0.016418	0.079104	0.046269	...	1	
5	0.084570	0.028190	0.065282	0.040059	...	0	
6	0.047297	0.023649	0.061937	0.085586	...	1	
7	0.066465	0.016616	0.058912	0.033233	...	0	
8	0.062338	0.015584	0.083117	0.059740	...	0	
9	0.081349	0.005952	0.075397	0.073413	...	1	
10	0.048295	0.009943	0.065341	0.117898	...	0	
11	0.106542	0.014953	0.065421	0.039252	...	0	
12	0.067947	0.022157	0.066470	0.042836	...	0	
13	0.084605	0.020804	0.054092	0.047157	...	1	
14	0.048889	0.006667	0.075556	0.091111	...	0	
15	0.047619	0.020408	0.051020	0.074830	...	0	
16	0.055156	0.028777	0.067946	0.064748	...	1	
17	0.053004	0.010601	0.067138	0.045936	...	0	
18	0.058003	0.028347	0.068905	0.073266	...	1	
19	0.077491	0.019680	0.059041	0.028290	...	0	
20	0.057789	0.019263	0.064489	0.067839	...	1	
21	0.082685	0.015564	0.051556	0.055447	...	1	
22	0.069124	0.024578	0.056836	0.061444	...	0	
23	0.063119	0.017327	0.075495	0.048267	...	0	
24	0.058499	0.011038	0.076159	0.044150	...	1	
25	0.039062	0.015625	0.070312	0.101562	...	0	
26	0.070376	0.009820	0.056465	0.049100	...	1	
27	0.022989	0.038314	0.076628	0.084291	...	0	
28	0.028708	0.033493	0.057416	0.057416	...	0	
29	0.048458	0.017621	0.061674	0.057269	...	0	

	dist_kinase2_glpl_norm	dist_glpl_mhd_norm	has_ploop	has_kinase2	\
0	0	0	0	1	
1	0	1	0	1	
2	1	1	1	4	
3	0	0	1	1	
4	0	1	1	3	
5	0	1	1	2	
6	1	1	1	4	
7	0	1	0	1	
8	0	0	0	0	
9	0	0	0	1	
10	0	1	0	1	
11	0	1	0	1	
12	0	1	0	1	
13	0	1	0	2	

14	0	0	0
15	0	1	1
16	1	0	1
17	0	1	0
18	0	1	1
19	0	1	1
20	0	0	1
21	0	1	1
22	0	1	1
23	0	0	0
24	0	1	1
25	0	0	1
26	0	1	1
27	0	0	1
28	0	0	0
29	0	0	0

	has_gpl	has_mhd	target	id	cluster
0	0	0	1	NP_001234459.3	POS_2155
1	2	0	1	NP_001306257.2	POS_1204
2	0	2	1	NP_001318035.2	POS_183
3	0	0	1	YEV57985.1	POS_2239
4	2	0	1	NP_001307923.1	POS_1146
5	2	0	1	NP_001307848.1	POS_1138
6	0	0	1	NP_001234202.1	POS_751
7	2	0	1	NP_001307817.1	POS_1161
8	0	0	1	NP_001355132.1	POS_1803
9	0	0	1	NP_001333606.1	POS_1483
10	0	0	1	NP_001308492.1	POS_1078
11	0	0	1	NP_001307892.1	POS_1402
12	2	0	1	NP_001307546.1	POS_1135
13	2	0	1	NP_001307224.1	POS_1040
14	0	0	1	NP_001234240.1	POS_1623
15	0	0	1	CAQ5401959.1	POS_2012
16	0	2	1	CAQ5402156.1	POS_179
17	0	0	1	CAQ5401373.1	POS_2035
18	0	0	1	CAQ5402324.1	POS_12
19	1	0	1	CAQ5401369.1	POS_892
20	0	1	1	CAQ5401367.1	POS_239
21	2	0	1	CAQ5401354.1	POS_424
22	0	0	1	CAQ5402419.1	POS_1183
23	1	1	1	CAQ5402454.1	POS_899
24	1	0	1	CAQ5401193.1	POS_698
25	0	0	1	CAQ5401117.1	POS_2363
26	0	1	1	CAQ5402682.1	POS_210
27	0	0	1	CAQ5400846.1	POS_2083
28	0	0	1	CAQ5400845.1	POS_2216
29	0	0	1	CAQ5400813.1	POS_2162

[30 rows x 450 columns]

Iniciando validação cruzada

Finalizada a validação cruzada para: RF

Finalizada a validação cruzada para: XGB

Finalizada a validação cruzada para: SVM

Cross-Validation:RF

test_recall: 0.7741 ± 0.0165

test_roc_auc: 0.9286 ± 0.0140

test_mcc: 0.7433 ± 0.0256

test_accuracy: 0.8654 ± 0.0129

test_precision: 0.9469 ± 0.0137

test_f1: 0.8518 ± 0.0149

Cross-Validation XGB

test_recall: 0.8249 ± 0.0105

test_roc_auc: 0.9355 ± 0.0122

test_mcc: 0.7580 ± 0.0177

test_accuracy: 0.8769 ± 0.0086

test_precision: 0.9207 ± 0.0134

test_f1: 0.8701 ± 0.0090

Cross-Validation SVM

test_recall: 0.8314 ± 0.0166

test_roc_auc: 0.9419 ± 0.0094

test_mcc: 0.7825 ± 0.0248

test_accuracy: 0.8887 ± 0.0126

test_precision: 0.9389 ± 0.0110

test_f1: 0.8818 ± 0.0140

	Modelo	Accuracy	Recall	Precision	F1	ROC_AUC	MCC
0	RF	0.865396	0.774073	0.946894	0.851774	0.928637	0.743273
1	XGB	0.876906	0.824942	0.920707	0.870131	0.935537	0.758003
2	SVM	0.888679	0.831362	0.938855	0.881822	0.941939	0.782510

Iniciando treinamento RF: 15:21:38

=====
Random Forest
=====

	precision	recall	f1-score	support
0	0.81	0.96	0.87	467
1	0.95	0.77	0.85	468
accuracy			0.86	935
macro avg	0.88	0.86	0.86	935

weighted avg 0.88 0.86 0.86 935

ROC-AUC : 0.9413605666282326
PR-AUC : 0.9529804142531714
MCC : 0.7394714881669767
F1 : 0.8490566037735849
Precision : 0.9473684210526315
Recall : 0.7692307692307693
Balanced Accuracy: 0.8632021083841213

Finalizado treinamento RF em: 15:21:40

Tempo decorrido: 0:00:01.813655

[SHAP] Inicializando análise de interpretabilidade para: Random Forest...
[SHAP] Gráficos salvos com sucesso em 'pipelineAB/'!

Iniciando treinamento XGBoost: 15:22:28

```
=====
XGBoost
=====
```

	precision	recall	f1-score	support
0	0.85	0.94	0.89	467
1	0.93	0.84	0.88	468
accuracy			0.89	935
macro avg	0.89	0.89	0.89	935
weighted avg	0.89	0.89	0.89	935

ROC-AUC : 0.9480499277073153
PR-AUC : 0.9580940548125865
MCC : 0.7750733756447838
F1 : 0.8796400449943758
Precision : 0.9287410926365796
Recall : 0.8354700854700855
Balanced Accuracy: 0.8856151283881477

Finalizado em XGBoost: 15:22:39

Tempo decorrido: 0:00:10.670345

[SHAP] Inicializando análise de interpretabilidade para: XGBoost...

[SHAP] Gráficos salvos com sucesso em 'pipelineAB/'!

Iniciando treinamento SVM: 15:22:40

=====

SVM

=====

	precision	recall	f1-score	support
0	0.85	0.91	0.88	467
1	0.91	0.84	0.87	468
accuracy			0.88	935
macro avg	0.88	0.88	0.88	935
weighted avg	0.88	0.88	0.88	935

ROC-AUC : 0.9438084518384304

PR-AUC : 0.9535831935130384

MCC : 0.7602027538742399

F1 : 0.8748615725359912

Precision : 0.9080459770114943

Recall : 0.844017094017094

Balanced Accuracy: 0.8791819945460202

Finalizado em SVM: 15:22:44

Tempo decorrido: 0:00:04.254952

[SVM] Calculando a importância por permutação dos atributos...

Top 20 features mais importantes para o SVM:

	feature	importance_mean
31	DIPEP_AM	0.005348
250	DIPEP_NL	0.004920
150	DIPEP_HL	0.004064
294	DIPEP_QQ	0.003422
68	DIPEP_DI	0.002567
300	DIPEP_QY	0.002567
428	RNBS_D	0.002353
430	DFG	0.002353
70	DIPEP_DL	0.002353
25	DIPEP_AF	0.002353
213	DIPEP_LP	0.002139
320	DIPEP_RY	0.002139
408	DIPEP_YI	0.002139

10	AAC_L	0.001925
390	DIPEP_WL	0.001925
253	DIPEP_NP	0.001925
196	DIPEP_KS	0.001925
398	DIPEP_WV	0.001925
426	RNBS_B	0.001925
144	DIPEP_HE	0.001711

Criando explainer shap...

PermutationExplainer explainer: 201it [1:37:33, 29.27s/it]

Shap SHAP: (200, 447)

[SVM] Gráfico salvo com sucesso em 'pipelineAB/svm_importancia_permutacao.png'!

Finalizado fase de treinamento em: 17:01:36

Tempo decorrido: 1:40:43.637343

Fim criação/treino dos modelos

<Figure size 500x500 with 0 Axes>

<Figure size 500x500 with 0 Axes>

<Figure size 500x500 with 0 Axes>

```
[11]: '''
Avaliação e Validação dos modelos
'''

print("\n\nIniciando Validação dos modelos\n")
# verificar importância dos atributos/feature para o Random Forest
importancia = pd.DataFrame({"feature":columns,"importance":rf_model.
    ↪feature_importances_})
importancia = importancia.sort_values("importance",ascending = False)
print("\nTop 20 features: Random Forest")
print(importancia.head(20))
# GRÁFICO
top = importancia.head(20)
plt.figure(figsize=(10,8))
plt.barh( top["feature"][::-1], top["importance"][::-1] )
plt.xlabel("Importância")
plt.ylabel("Feature")
# plt.title("Top 20 Features - Random Forest")
```

```

plt.tight_layout()
plt.savefig(
    f"{OUTPUT_DIR}/rf_importancia_variaveis.png",
    bbox_inches="tight"
)
plt.close()

# verificar importância dos atributos/feature para o XGBoost
importancia = pd.DataFrame({"feature":columns, "importance":xgb_model.
    ↪feature_importances_})
importancia = importancia.sort_values("importance", ascending = False)
print("\nTop 20 features: XGBoost")
print(importancia.head(20))
# GRÁFICO
top = importancia.head(20)
plt.figure(figsize=(10,8))
plt.barh(top["feature"][::-1], top["importance"][::-1])
plt.xlabel("Importância")
plt.ylabel("Feature")
# plt.title("Top 20 Features - XGBoost")
plt.tight_layout()
plt.savefig(
    f"{OUTPUT_DIR}/xgb_importancia_variaveis.png",
    bbox_inches="tight"
)
plt.close()

print("\nCOMPARAÇÃO MODELOS")
print(pd.DataFrame(results))

results_df = pd.DataFrame(results)
results_df.to_csv(
    f"{OUTPUT_DIR}/metricas_comparacao.csv",
    index=False
)

metrics = [
    "ROC_AUC",
    "F1",
    "Precision",
    "Recall",
    "Balanced_Accuracy"
]

for metric in metrics:
    plt.figure(figsize=(8,5))
    plt.bar(

```

```

        results_df["Modelo"],
        results_df[metric]
    )

    plt.ylim(0, 1)
    plt.ylabel(metric)
    # plt.title(f"Comparação dos Modelos - {metric}")
    for i, v in enumerate(results_df[metric]):
        plt.text(i, v + 0.01, f"{v:.3f}", ha='center')

    plt.savefig(
        f"{OUTPUT_DIR}/comparacao_{metric}.png",
        bbox_inches="tight"
    )
    plt.close()

modelos = {
    "Random Forest": rf_model,
    "XGBoost": xgb_model,
    "SVM": svm_model
}

plt.figure(figsize=(7,7))
for nome, model in modelos.items():
    probs = model.predict_proba(X_test)[: ,1]
    fpr, tpr, _ = roc_curve(y_test, probs)
    roc_auc = auc(fpr, tpr)
    plt.plot(
        fpr,
        tpr,
        label=f"{nome} AUC={roc_auc:.3f}"
    )
plt.plot([0,1],[0,1], '--')
plt.xlabel("FPR")
plt.ylabel("TPR")
# plt.title("Comparação ROC")
plt.legend()
plt.savefig(f"{OUTPUT_DIR}/roc_comparativa.png")
plt.close()

print("Fim avaliação / validação modelos ")

```

Iniciando Validação dos modelos

Top 20 features: Random Forest
 feature importance

10	AAC_L	0.066028
250	DIPEP_NL	0.045765
1	AAC_A	0.031771
204	DIPEP_LE	0.027663
216	DIPEP_LS	0.022592
190	DIPEP_KL	0.019302
330	DIPEP_SL	0.018961
209	DIPEP_LK	0.017400
214	DIPEP_LQ	0.015220
433	NUM_NLR_MOTIFS	0.013913
70	DIPEP_DL	0.013858
90	DIPEP_EL	0.013059
0	protein_length	0.013010
21	DIPEP_AA	0.012556
203	DIPEP_LD	0.012520
215	DIPEP_LR	0.011955
150	DIPEP_HL	0.011942
217	DIPEP_LT	0.011328
121	DIPEP_GA	0.006883
38	DIPEP_AV	0.006644

Top 20 features: XGBoost

	feature	importance
10	AAC_L	0.047701
437	dist_ploop_kinase2	0.015063
250	DIPEP_NL	0.014361
1	AAC_A	0.014132
444	has_kinase2	0.011982
438	dist_kinase2_glp1	0.010618
441	dist_kinase2_glp1_norm	0.008700
422	KINASE2	0.008525
0	protein_length	0.007229
433	NUM_NLR_MOTIFS	0.006851
204	DIPEP_LE	0.005938
6	AAC_G	0.005464
445	has_glp1	0.005317
4	AAC_E	0.005051
205	DIPEP_LF	0.005028
424	MHD	0.004883
322	DIPEP_SC	0.004410
330	DIPEP_SL	0.004384
11	AAC_M	0.004271
209	DIPEP_LK	0.004115

COMPARAÇÃO MODELOS

	Modelo	ROC_AUC	F1	Precision	Recall	Balanced_Accuracy \
0	Random Forest	0.941361	0.849057	0.947368	0.769231	0.863202
1	XGBoost	0.948050	0.879640	0.928741	0.835470	0.885615

2 SVM 0.943808 0.874862 0.908046 0.844017 0.879182

MCC

0 0.739471

1 0.775073

2 0.760203

Fim avaliação / validação modelos

```
[ ]: '''
Funções para rodar a aplicação
'''

# motifs = {
#     "P_LOOP": r"[A-Z]G[A-Z]{4}GK[ST]",
#     "KINASE2": r"[LIVM]{3,5}DD",
#     "RNBS_A": r"FDL",
#     "RNBS_B": r"G[RK]G",
#     "RNBS_C": r"[A-Z]{2}P[A-Z]{2}W",
#     "GLPL": r"GLPL",
#     "RNBS_D": r"CFL",
#     "MHD": r"MHD",
# }
motifs = {
    # =====
    # NLR (CNL/TNL)
    # =====
    "P_LOOP": r"[AGST]{3,5}GK[TS]",
    "KINASE2": r"[ALIVMFYW]{3,5}DD[VILW][WFY]",
    "GLPL": r"G[LVI][PAST][LIVAF]",
    "MHD": r"[MVIL][HN][DE][VILAST]",
    "RNBS_A": r"F.DL.{0,3}AW",
    "RNBS_B": r"G[SA]{2,6}T",
    "RNBS_C": r"Y.[VIL]{1,3}[LS]",
    "RNBS_D": r"C.{2,8}P",

    # =====
    # Protein Kinase (KIN/RLK)
    # =====
    "HRD": r"HRD[LIVMFY]",
    "DFG": r"DFG",

    # =====
    # LRR (RLP/RLK)
    # =====
    "LRR1": r"L...L...L...N",
    "LRR2": r"L[A-Z]{2}L[A-Z]{2}L[A-Z]{2}[NCT]"
}
```

```

df_validacao = []

def validar_motifs(df,planta,modelo):
    print("\nVALIDAÇÃO BIOLÓGICA\n")
    counts = {}
    for motif_name, pattern in motifs.items():
        counts[motif_name] = 0
        for seq in df["protein_seq"]:
            if re.search(pattern, seq):
                counts[motif_name] += 1
    for k, v in counts.items():
        print(k, ":", v)

    counts['planta'] = planta
    counts['modelo'] = modelo

    df_validacao.append(counts)
    df_counts = pd.DataFrame([counts])
    df_counts.to_csv(
        f"{OUTPUT_DIR}/validacao_biologica_{modelo}_{planta}.csv",
        index=False
    )

def rodar_transdecoder(fasta_nt):
    cmd1 = [
        "TransDecoder.LongOrfs",
        "-t",
        fasta_nt,
        "--output_dir",
        f"{OUTPUT_DIR}"
    ]
    with open(fasta_nt+"_output1.txt", "a", encoding="utf-8") as log_file:
        subprocess.run(cmd1,
                        stdout=log_file,
                        stderr=subprocess.STDOUT,
                        text=True)

    cmd2 = [
        "TransDecoder.Predict",
        "-t",
        fasta_nt,
        "--output_dir",
        f"{OUTPUT_DIR}"
    ]

```

```

with open(fasta_nt+"_output2.txt", "a", encoding="utf-8") as log_file:
    subprocess.run(cmd2,
                    stdout=log_file,
                    stderr=subprocess.STDOUT,
                    text=True)
pep_file = fasta_nt + ".transdecoder.pep"
return pep_file

def predizer_transcritos(fasta_nt,model,nmodel, output, planta):
    probabilidade = 0.85
    candidatos = []
    pep_file = rodar_transdecoder(fasta_nt)
    for record in SeqIO.parse(fasta_nt+".transdecoder.pep", "fasta"):
        protein = str(record.seq)
        feats = extrair_features_proteina(protein)
        if feats is None:
            continue
        X = scaler.transform([feats])
        prob = model.predict_proba(X)[0][1]
        candidatos.append({
            "id": record.id,
            "descricao": record.description,
            "protein_length": len(protein),
            "probabilidade_resistencia": prob,
            "protein_seq": protein
        })

    candidatos_df = pd.DataFrame(candidatos)
    candidatos_df = candidatos_df.sort_values(
        "probabilidade_resistencia",
        ascending=False
    )

    # top = candidatos_df.head(40)
    top = candidatos_df[candidatos_df["probabilidade_resistencia"] >=
    ↪probabilidade]
    print("\nTOP CANDIDATOS com probabilidade de >= "+probabilidade+" -
    ↪"+fasta_nt+", MODELO: "+nmodel)
    print(top[[
        "id",
        "descricao",
        "protein_length",
        "probabilidade_resistencia"
    ]])

    validar_motifs(top,planta,nmodel)

```

```

candidatos_df.to_csv(
    f"{OUTPUT_DIR}/{output}",
    index=False
)

```

```

[ ]: '''
Aplicação - Modelo XGBoost
'''

print("\n\n=====Iniciando Aplicação: Predição de_
↳ Genes===== \n\n")
print("\n\n Predição Modelo XGBoost")
inicio = datetime.now()
print("\n\nIniciando predição ARABIDOPSIS: ", inicio.strftime("%H:%M:%S"))

print("\nPredição para ARABIDOPSIS\n")
predizer_transcritos(transcriptoma_arabidopsis_fasta,
↳ xgb_model, 'XGBoost', output="predicoes_arabidopsis_xgboost.
↳ csv", planta="Arabidopsis thaliana")

fim = datetime.now()
print("\n\nFinalizado predição ARABIDOPSIS:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

```

```

[ ]: print("\nPredição para MELONGENA\n")
inicio = datetime.now()
print("\n\nIniciando predição MELONGENA: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_melongena_fasta,
↳ xgb_model, 'XGBoost', output="predicoes_melongena_xgboost.csv", planta="Solanum_
↳ melongena")
fim = datetime.now()
print("\n\nFinalizado predição MELONGENA:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

```

```

[ ]: print("\nPredição para TUBEROSUM\n")
inicio = datetime.now()
print("\n\nIniciando predição TUBEROSUM: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_tuberosum_fasta,
↳ xgb_model, 'XGBoost', output="predicoes_tuberosum_xgboost.csv", planta="Solanum_
↳ tuberosum")
fim = datetime.now()
print("\n\nFinalizado predição TUBEROSUM:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

```

```

[ ]: print("\nPredição para PHYSALIS\n")
inicio = datetime.now()
print("\n\nIniciando predição PHYSALIS: ", inicio.strftime("%H:%M:%S"))

```



```

predizer_transcritos(transcriptoma_physalis_fasta,
    ↳xgb_model, 'XGBoost', output="predicoes_physalis_xgboost.csv", planta="Physalis
    ↳peruviana")
fim = datetime.now()
print("\n\nFinalizado predição TUBEROSUM:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
print("\nArquivo salvo:")
print(f"{OUTPUT_DIR}/predicoes_xgboost.csv")

```

```
[ ]: print(df_validacao)
```

```

[ ]: '''
    Aplicação - Modelo Random Forest
    '''

print("\n\n Predição Modelo RF")
print("\nPredição para ARABIDOPSIS\n")
inicio = datetime.now()
print("\n\nIniciando predição ARABIDOPSIS: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_arabidopsis_fasta,
    ↳rf_model, "RF", output="predicoes_arabidopsis_rf.csv", planta="Arabidopsis
    ↳thaliana")
print("\n\nFinalizado predição ARABIDOPSIS:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

```

```

[ ]: print("\nPredição para MELONGENA\n")
inicio = datetime.now()
print("\n\nIniciando predição MELONGENA: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_melongena_fasta,
    ↳rf_model, 'RF', output="predicoes_melongena_rf.csv", planta="Solanum melongena")
print("\n\nFinalizado predição MELONGENA:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

```

```

[ ]: print("\nPredição para TUBEROSUM\n")
inicio = datetime.now()
print("\n\nIniciando predição TUBEROSUM: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_tuberosum_fasta,
    ↳rf_model, "RF", output="predicoes_tuberosum_rf.csv", planta="Solanum tuberosum")
print("\n\nFinalizado predição TUBEROSUM:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

```

```

[ ]: print("\nPredição para PHYSALIS\n")
inicio = datetime.now()
print("\n\nIniciando predição PHYSALIS: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_physalis_fasta,
    ↳rf_model, "RF", output="predicoes_physalis_rf.csv", planta="Physalis peruviana")
print("\n\nFinalizado predição PHYSALIS:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

```

```
print("\nArquivo salvo:")
print(f"{OUTPUT_DIR}/predicoes_rf.csv")
```

```
[ ]: '''
Aplicação - Modelo SVM
'''

print("\n\n Predição Modelo SVM")
print("\nPredição para ARABIDOPSIS\n")
inicio = datetime.now()
print("\n\nIniciando predição ARABIDOPSIS: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_arabidopsis_fasta,
    ↪svm_model, "SVM", output="predicoes_arabidopsis_svm.csv", planta="Arabidopsis_
    ↪thaliana")
print("\n\nFinalizado predição ARABIDOPSIS:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
[ ]: print("\nPredição para MELONGENA\n")
inicio = datetime.now()
print("\n\nIniciando predição MELONGENA: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_melongena_fasta,
    ↪svm_model, 'SVM', output="predicoes_melongena_svm.csv", planta="Solanum_
    ↪melongena")
print("\n\nFinalizado predição MELONGENA:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
[ ]: print("\nPredição para TUBEROSUM\n")
inicio = datetime.now()
print("\n\nIniciando predição TUBEROSUM: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_tuberosum_fasta,
    ↪svm_model, "SVM", output="predicoes_tuberosum_svm.csv", planta="Solanum_
    ↪tuberosum")
print("\n\nFinalizado predição TUBEROSUM:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
[ ]: print("\nPredição para PHYSALIS\n")
inicio = datetime.now()
print("\n\nIniciando predição PHYSALIS: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_physalis_fasta,
    ↪svm_model, "SVM", output="predicoes_physalis_svm.csv", planta="Physalis_
    ↪peruviana")
print("\n\nFinalizado predição PHYSALIS:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

print("\nArquivo salvo:")
print(f"{OUTPUT_DIR}/predicoes_svm.csv")
```

```
[ ]: print(df_validacao)
```

```
[ ]: df_copia = df_validacao.copy()
df_inicial = pd.DataFrame(df_validacao)

df_validacao = df_inicial.melt(
    id_vars=["planta", "modelo"],
    var_name="motivo",
    value_name="contagem"
)
print(df_validacao.head())

df_validacao.to_csv(f"{OUTPUT_DIR}/df_validacao.csv", index=False)
```

```
[ ]: #Scatterplot
plt.figure(figsize=(12,6))
sns.scatterplot(
    data=df_validacao,
    x="motivo",
    y="contagem",
    hue="planta",
    style="modelo",
    s=200
)
# plt.title( "Validação biológica dos motivos conservados")
plt.ylabel("Número de ocorrências")
plt.xlabel("Motivo")
plt.legend(
    bbox_to_anchor=(1.05,1),
    loc="upper left"
)
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/Validacao_biologica_dos_motivos_conservados.png")
plt.close()
# plt.show()

#Bubble Chart
plt.figure(figsize=(12,6))
sns.scatterplot(
    data=df_validacao,
    x="motivo",
    y="modelo",
    hue="planta",
    size="contagem",
    sizes=(50,600)
)
# plt.title("Distribuição dos motivos por modelo e espécie")
```

```

plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/distribuicao_dos_motivos_por_modelo_especie.png")
plt.close()
# plt.show()

#Facetas por planta
g = sns.catplot(
    data=df_validacao,
    x="motivo",
    y="contagem",
    hue="modelo",
    col="planta",
    kind="bar",
    height=5,
    aspect=1.2
)
g.set_xticklabels(rotation=45)
plt.savefig(f"{OUTPUT_DIR}/facetas_por_planta.png")
plt.close()
# plt.show()

#por planta

#Physalis peruviana
df_physalis = df_validacao[df_validacao["planta"] == "Physalis peruviana"]
plt.figure(figsize=(12, 6))
sns.set_theme(style="whitegrid")
sns.lineplot(
    data=df_physalis,
    x="motivo",
    y="contagem",
    hue="modelo",
    style="modelo",
    markers=True,      # Ativa os marcadores nos pontos
    dashes=False,      # Mantém as linhas contínuas
    linewidth=2,       # Espessura da linha
    markersize=10      # Tamanho dos pontos de dispersão
)

# plt.title("Comparação dos Modelos para Physalis peruviana", fontsize=14,
#           fontweight="bold", pad=15)
plt.xlabel("Motivos Conservados", fontsize=12, labelpad=10)
plt.ylabel("Número de Ocorrências (Contagem)", fontsize=12, labelpad=10)
plt.xticks(rotation=45, ha="right")
plt.legend(title="Modelos", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()

```

```

plt.savefig(f"{OUTPUT_DIR}/comparacao_modelos_physalis.png", dpi=300)
plt.close()

#Solanum melongena
df_melongena = df_validacao[df_validacao["planta"] == "Solanum melongena"]
plt.figure(figsize=(12, 6))
sns.set_theme(style="whitegrid")
sns.lineplot(
    data=df_melongena,
    x="motivo",
    y="contagem",
    hue="modelo",
    style="modelo",
    markers=True,      # Ativa os marcadores nos pontos
    dashes=False,      # Mantém as linhas contínuas
    linewidth=2,       # Espessura da linha
    markersize=10      # Tamanho dos pontos de dispersão
)

# plt.title("Comparação dos Modelos para Solanum melongena", fontsize=14,
#           ↪fontweight="bold", pad=15)
plt.xlabel("Motivos Conservados", fontsize=12, labelpad=10)
plt.ylabel("Número de Ocorrências (Contagem)", fontsize=12, labelpad=10)
plt.xticks(rotation=45, ha="right")
plt.legend(title="Modelos", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/comparacao_modelos_melongena.png", dpi=300)
plt.close()

#Solanum tuberosum
df_tuberosum = df_validacao[df_validacao["planta"] == "Solanum tuberosum"]
plt.figure(figsize=(12, 6))
sns.set_theme(style="whitegrid")
sns.lineplot(
    data=df_tuberosum,
    x="motivo",
    y="contagem",
    hue="modelo",
    style="modelo",
    markers=True,      # Ativa os marcadores nos pontos
    dashes=False,      # Mantém as linhas contínuas
    linewidth=2,       # Espessura da linha
    markersize=10      # Tamanho dos pontos de dispersão
)

# plt.title("Comparação dos Modelos para Solanum tuberosum", fontsize=14,
#           ↪fontweight="bold", pad=15)

```

```

plt.xlabel("Motivos Conservados", fontsize=12, labelpad=10)
plt.ylabel("Número de Ocorrências (Contagem)", fontsize=12, labelpad=10)
plt.xticks(rotation=45, ha="right")
plt.legend(title="Modelos", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/comparacao_modelos_tuberosum.png", dpi=300)
plt.close()

#Arabidopsis thaliana
df_arabidopsis = df_validacao[df_validacao["planta"] == "Arabidopsis thaliana"]
plt.figure(figsize=(12, 6))
sns.set_theme(style="whitegrid")
sns.lineplot(
    data=df_arabidopsis,
    x="motivo",
    y="contagem",
    hue="modelo",
    style="modelo",
    markers=True,      # Ativa os marcadores nos pontos
    dashes=False,      # Mantém as linhas contínuas
    linewidth=2,       # Espessura da linha
    markersize=10      # Tamanho dos pontos de dispersão
)

# plt.title("Comparação dos Modelos para Arabidopsis thaliana", fontsize=14,
#           fontweight="bold", pad=15)
plt.xlabel("Motivos Conservados", fontsize=12, labelpad=10)
plt.ylabel("Número de Ocorrências (Contagem)", fontsize=12, labelpad=10)
plt.xticks(rotation=45, ha="right")
plt.legend(title="Modelos", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/comparacao_modelos_arabidopsis.png", dpi=300)
plt.close()

#Heatmap
df_validacao["grupo"] = (
    df_validacao["planta"]
    + "_"
    + df_validacao["modelo"]
)

pivot = df_validacao.pivot_table(
    index="motivo",
    columns="grupo",
    values="contagem",
    fill_value=0
)

```

```

plt.figure(figsize=(14, 8))
sns.heatmap(
    pivot,
    annot=True,
    fmt="g",
    cmap="YlGnBu",
    linewidths=0.5,
    cbar_kws={'label': 'Contagem'} # Nomeia a barra de cores lateral
)

# plt.title("Distribuição dos Motivos Conservados por Planta e Modelo",
#           ↪fontsize=14, fontweight="bold", pad=15)
plt.ylabel("Motivo", fontsize=12)
plt.xlabel("Grupo (Planta & Modelo)", fontsize=12)
plt.xticks(rotation=45, ha="right", fontsize=10)
plt.yticks(rotation=0, fontsize=10)
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/heatmap_distribuicao_dos_motivos_conservados.
           ↪png", dpi=300)
plt.close()
# plt.show()

print("==== Fim do Pipeline =====")

```